



INSTITUTO SUPERIOR POLITÉCNICO DE TECNOLOGIAS E CIÊNCIAS

**INSTITUTO SUPERIOR POLITÉCNICO DE TECNOLOGIAS E
CIÊNCIAS DEPARTAMENTO DE ENGENHARIAS CURSO
DE ENGENHARIA INFORMÁTICA**

RELATÓRIO DE PROJECTO

Sistema de Gestão de Frotas

Frota Simba Wambau

Programação II — Engenharia Informática
Turma EINF4_T3 | Docente: Sílvia António

Grupo: 15

Estefani Famorosa — N° 20240217
Jorje Veloso — N° 20241500
Josemar Chipandeca — N° 20242447

**LUANDA
2025/2026**





1. Identificação do Projecto

Campo	Detalhe
Disciplina	Programação II
Curso	Engenharia Informática — EINF4_T3
Docente	Sílvia António
Ano Lectivo	2025/2026
Repositório	github.com/Safani-gou/Gestao_de_Frotas_2PP_Prog-II
Linguagem	Java (JDK 17+)
Membros do Grupo	Estefani Famosa (Nº 20240217) Jorje Veloso (Nº 20241500) Josemar Chipandeca (Nº 20242447)

2. Introdução

O presente relatório documenta o desenvolvimento do projecto "Sistema de Gestão de Frotas — Frota Simba Wambau", realizado no âmbito da disciplina de Programação II. O sistema tem como objectivo gerir uma frota de veículos de aluguer, permitindo registar, alugar, devolver e controlar a manutenção de automóveis, bem como gerir os clientes associados.

O projecto foi implementado integralmente em Java, recorrendo a programação orientada a objectos, colecções dinâmicas, persistência em ficheiros de texto e validação de dados por expressões regulares. A interacção com o utilizador é feita através de menus em terminal, com dois perfis distintos de acesso: Gestor e Operário.

3. Objectivos

O objectivo principal do projecto é consolidar os conhecimentos de programação orientada a objectos em Java, aplicando os seguintes conceitos obrigatórios definidos no enunciado:

- Estruturas de selecção: if/else e switch
- Estruturas de repetição: for e do/while
- ArrayList para gestão de colecções de objectos
- Registos — classes com atributos e métodos
- Manipulação de Strings
- Funções/métodos organizados por responsabilidade
- Ficheiros para persistência de dados (leitura e escrita em .txt)



Para além dos conceitos base, o projecto implementa estruturas adicionais como Queue (fila de manutenção), Iterator (remoção segura de listas) e expressões regulares para validação de entradas.

4. Arquitectura do Sistema

4.1. Estrutura de Classes

O sistema é organizado em 7 classes com responsabilidades bem delimitadas:

Classe	Responsabilidade	Estruturas Usadas	Principais Métodos
Main	Ponto de entrada. Menus do Gestor e Operário.	do/while, switch	main()
FrotaService	Toda a lógica de negócio e persistência em ficheiros.	ArrayList, Queue, Iterator, BufferedReader, PrintWriter	adicionarCarro(), alugarCarro(), devolverCarro(), enviarParaManutencao(), concluirManutencao(), relatorioEstatistico()
Carro	Modelo do veículo com estado de disponibilidade e manutenção.	boolean, String	toFileString(), fromFileString(), getters/setters
Cliente	Dados pessoais e estado activo/inactivo.	boolean, String	formatoString(), leFormatoString(), isActive()
Aluguer	Registo de uma operação de aluguer com datas.	String	toFileString(), fromFileString(), setDataDevolucao()
Manutencao	Entrada na fila de manutenção.	LocalDate	getMatricula(), getDescricao(), getDataEntrada()
Validar	Validação de entradas do utilizador via regex.	String.matches()	matricula(), nomeFistLastClie(), n_bilheteClie(), telefoneClie(), ano()

4.2. Diagrama de Responsabilidades

Separação de responsabilidades:



- **Josemar Chipandeca:** Main.java — interface com o utilizador (menus, leitura de input) e Carro — Modelo de dados (entidade)
- **Estefani Famorosa:** FrotaService.java — lógica de negócio e persistência (camada de serviço), Cliente — Modelo de dados (entidade) e Validar.java — validação de todas as entradas do utilizador
- **Jorje Veloso:** Aluguer, Manutencao — modelos de dados (entidades)

Esta separação garante que cada classe tem uma única responsabilidade, facilitando a manutenção e extensão do código.

5. Funcionamento do Sistema

5.1. Perfis de Acesso

O menu principal apresenta três opções: Gestor, Operário e Sair. Cada perfil tem acesso a funcionalidades distintas:

- Gestor — acesso completo à gestão de carros, clientes, manutenção e relatórios estatísticos.
- Operário — atendimento ao cliente: registo de carros e clientes, aluguer, devolução, consulta de disponibilidade e histórico.

5.2. Gestão de Carros

- Registrar carro: matrícula (validada por regex), marca, modelo e ano.
- Listar todos os carros da frota com estado completo.
- Listar apenas carros disponíveis para aluguer.
- Pesquisar carro por matrícula.
- Remover carro (apenas se não estiver alugado nem em manutenção).

5.3. Gestão de Clientes

- Registrar cliente: nome, idade, Nº BI, endereço e telefone (todos validados).
- Listar todos os clientes.
- Listar clientes activos (com carro actualmente alugado).
- Remover cliente (apenas se não tiver aluguer activo).

5.4. Processo de Aluguer e Devolução

O aluguer valida múltiplas condições antes de ser registado:

1. Verificar se o carro existe na frota.
2. Verificar se o carro está disponível (não alugado).
3. Verificar se o carro não está em manutenção.



4. Verificar se o cliente existe no sistema.
5. Verificar se o cliente não tem já um carro alugado.
6. Registar o aluguer com data/hora actual e guardar em ficheiro.

A devolução regista a data de devolução no histórico, liberta o carro e desactiva o estado do cliente.

5.5. Fila de Manutenção

A manutenção usa uma Queue<Manutencao> implementada com LinkedList, garantindo ordem FIFO — o primeiro carro a entrar é o primeiro a sair:

- Enviar para manutenção: carro é retirado de disponível e adicionado à fila.
- Consultar fila: lista todos os carros aguardando manutenção.
- Concluir manutenção: método poll() remove o primeiro da fila e o carro volta a disponível.

5.6. Relatório Estatístico

O relatório percorre a lista de carros e calcula:

- Total de veículos na frota
- Veículos disponíveis
- Veículos alugados
- Veículos em manutenção
- Taxa de utilização: $(\text{alugados} / \text{total}) \times 100\%$

6. Persistência de Dados

Todos os dados são automaticamente guardados em ficheiros .txt após cada operação que os altere, e carregados ao iniciar o programa. O sistema usa 4 ficheiros:

Ficheiro	Conteúdo	Formato de Cada Linha
carros.txt	Todos os veículos da frota	matricula;marca;modelo;ano;disponivel;emManutencao;clienteBi
clientes.txt	Clientes registados	nome;idade;n_bi;endereco;telefone;activo
alugueres.txt	Histórico de alugueres	matricula;n_BI;dataAluguer;dataDevolucao
manutencoes	Fila de	matricula;descricao;dataEntrada



.txt	manutenção o activa	
------	------------------------	--

A leitura é feita com `BufferedReader` e `FileReader`. A escrita usa `PrintWriter` e `FileWriter`. Cada objecto é serializado numa única linha com campos separados por ponto e vírgula (;), e desserializado com `split(";", -1)`.

7. Validação de Dados

A classe `Validar` centraliza toda a validação de entradas usando expressões regulares (`String.matches()`). O utilizador não avança enquanto o dado introduzido não for válido — cada campo tem um ciclo `while` dedicado:

Campo	Regex	Exemplo Válido
Matrícula	<code>[A-Z]{2}-\d{2}-\d{2}-[A-Z]{2}</code>	LD-23-45-AA
Nome	<code>[A-Z]+[a-zÀ-ÿ]+\s+[A-Z]+[a-zÀ-ÿ]+</code>	João Silva
Nº BI	<code>\d{9}[A-Z][A-Z]\d{3}</code>	123456789LA001
Telefone	<code>9\d{8}</code>	923456789
Idade	<code>\d{2}</code>	25
Ano	<code>\d{4}</code>	2020

8. Conceitos Obrigatórios Aplicados

A tabela seguinte relaciona cada conceito exigido no enunciado com a sua aplicação concreta no projecto:

Conceito do Enunciado	Aplicação no Projecto
Estruturas de selecção (if/else, switch)	switch nos menus de Gestor e Operário (Main.java); if/else para validar estado dos carros e clientes antes de alugar ou remover (FrotaService.java).
Estruturas de repetição (for, do/while)	do/while mantém o utilizador dentro de cada menu; for-each percorre ArrayList de carros, clientes e alugueres para listar, pesquisar e filtrar.
ArrayList	Três ArrayList em FrotaService: carros, clientes, alugueres. Permitem adicionar, remover e iterar objectos dinamicamente.
Registos (classes com	Classes Carro, Cliente, Aluguer e Manutencao



atributos e métodos)	encapsulam atributos privados com getters/setters e métodos de serialização.
Manipulação de Strings	Expressões regulares em Validar.java; serialização com separador ';' nos métodos toFileString()/fromFileString(); formatação de datas com DateTimeFormatter.
Funções organizadas por responsabilidade	Lógica de negócio em FrotaService, interface em Main, validação em Validar, cada entidade na sua classe.
Ficheiros (.txt)	4 ficheiros persistidos automaticamente. Carregados ao iniciar com BufferedReader/FileReader; guardados após cada operação com PrintWriter/FileWriter.

9. Problemas Identificados e Correções

Durante a revisão do código foram identificados e corrigidos 7 problemas que comprometiam o correcto funcionamento do sistema:

#	Ficheiro	Problema	Correcção
1	Carro.java	emManutencao não era serializado — ao reiniciar, carros em manutenção ficavam disponíveis.	toFileString() inclui emManutencao; fromFileString() lê 7 campos.
2	Validar.java	Regex da matrícula aceitava separadores inválidos e não forçava exactamente 2 letras por grupo.	Novo regex: [A-Z]{2}-\d{2}-\d{2}-[A-Z]{2}
3	FrotaService.java	Dois Scanner criados sobre System.in causavam comportamento imprevisível.	FrotaService recebe o Scanner via construtor. Partilhado e único.
4	FrotaService.java	removerCliente() e removerCarro() modificavam a lista durante iteração for-each → ConcurrentModificationException.	Substituído por Iterator com it.remove().
5	FrotaService.java	concluirManutencao() fazia chamada recursiva se o carro não fosse encontrado → risco de StackOverflowError.	Eliminada a recursão. Adicionada mensagem de aviso. Ficheiros guardados correctamente.
6	Main.java	Faltava break e case "Operario": após o do-while do Gestor — o fluxo caía para o código do Operário.	Adicionado break; e case "Operario": na posição correcta.
7	Main.java	case 8 (Manutenção) no	Adicionado break;



		Operário não tinha break → fall-through encerrava a sessão involuntariamente.	após o do-while do case 8.
--	--	---	----------------------------

Após as correcções, todos os ficheiros foram verificados e confirmados como correctos.

10. Dificuldades Encontradas

- Gestão de dois Scanner sobre o mesmo System.in — resolvida passando o scanner por construtor.
- Serialização incompleta do campo em Manutencao — o estado de manutenção era perdido ao reiniciar o programa.
- Modificação de listas durante iteração — substituição de for-each por Iterator.
- Estrutura dos menus com switch aninhados — o fall-through involuntário entre cases exigiu revisão cuidadosa dos breaks.
- Organização dos ficheiros de persistência para que reflectam sempre o estado actual do sistema após cada operação.

11. Conclusão

O projecto implementa com sucesso um sistema de gestão de frotas em Java, cobrindo todos os requisitos definidos no enunciado. Foram aplicados os sete conceitos obrigatórios — selecção, repetição, ArrayList, registos, Strings, métodos e ficheiros — e adicionadas estruturas complementares como Queue, Iterator e expressões regulares.

Os sete bugs identificados durante a revisão foram corrigidos, tornando o sistema estável e funcionalmente correcto. O projecto demonstra capacidade de organização do código por responsabilidades, controlo de fluxo complexo com menus aninhados e integração completa de persistência em ficheiros.

As principais lições aprendidas incluem a importância de serializar todos os campos relevantes, de usar Iterator para modificações seguras em listas, e de testar cuidadosamente cada break em estruturas switch aninhadas.

GitHub: https://github.com/Safani-gou/Gestao_de_Frotas_2PP_e_Atualizado-ExameFinal_Prog-II.git

Referências

- Deitel, P. & Deitel, H. — Java: Como Programar (10ª Edição). Pearson.
- Documentação oficial Java SE — <https://docs.oracle.com/en/java/>
- Repositório do projecto — https://github.com/Safani-gou/Gestao_de_Frotas_2PP_Prog-II